

PROMPT 01: RUST CORE — Operadores, Red Neuronal y Durable Object

****Contexto verificable (2026-10-02)****

- Repo: `RigohI/auto-healing-agent`, branch `main`, HEAD `f7522c8`
- Stack: Rust + workers-rs 0.8 + Cloudflare Workers + KV + Durable Objects (SQLite)
- Arquitectura: `no\_std` + alloc en `repair\_nn\_core`, Worker como orquestador (NO computación)
- SoT: `docs/GOVERNANCE.md` (MIN_CONFIDENCE=0.55, MAX_RISK=0.45)
- Restricción: ****Ningún LLM****, ningún código generado por la red. Solo `RepairAction` → operador determinista.

ALCANCE (Fases 0-4)

FASE 0 — Reconciliar PRs (BLOQUEANTE)

****Acciones:****

```
gh pr list --state draft --json number,title,isDraft,statusCheckRollup for N in $(gh pr list --state draft --json number -q '[][.number]'); do gh pr view $N --json files,title,additions,deletions,changedFiles gh pr diff $N gh run list --limit 5 --workflow "$(gh pr view $N --json workflowRun -q '.workflowRun.name')" done
```

****Criterio:****

- Si PR modifica `worker/Cargo.toml` o `Cargo.lock`: ****CLOSE**** (el Worker usa `worker = "0.8"` ya actualizado)
- Si PR añade `Cargo.lock` sin cambios funcionales: ****MERGE**** (consistencia de dependencias)
- Si PR toca `worker/src/lib.rs` con lógica que compile con `main`: ****REBASE**** y alinear

****Decisión final:**** Documentar en `docs/DISCREPANCIES.md` con justificación por PR.

FASE 1 — Allowlist de Operadores Rust

****Problema:**** Los 13 operadores actuales apuntan a Node/TS (`package.json`, `tsconfig.json`, `vercel.json`).

****Allowlist propuesto (15 operadores):****

```
// crates/repair_types/src/lib.rs pub enum OperatorId { NoOp = 0, // Sin acción
DependencyRepair = 1, // Cargo.toml: dependencia ausente (sin bump mayor) VersionPin = 2, // Cargo.toml + Cargo.lock: fijar versión conocida-buena LockfileRefresh = 3, // Cargo.lock: refrescar dentro de rangos FeatureFlagRepair = 4, // Cargo.toml [features]: feature faltante ImportPathFix = 5, // mod/use: especificador → ruta existente WorkspaceMemberFix = 6, // Cargo.toml raíz: members[] RustToolchainPin = 7, // rust-toolchain.toml BuildScriptFix = 8, // build.rs TypeAnnotationFix = 9, // anotación o ensanchamiento de tipo ClippyLintFix = 10, // allow de lint con justificación TestRepair = 11, // fixture o aserción (NUNCA la intención) EnvVarRepair = 12, // advisory (NUNCA inventa secretos) UnsafeBlockReview = 13, // advisory, escala a humano Unknown = 255, // Fallback explícito } pub const OPERATOR_COUNT: usize = 14; // 0-13 = 14 variantes
```

****Validación:****

- Cada operador debe tener ****incidentes reales de Rust**** que lo justifiquen
- Ejemplo para `DependencyRepair`: `error[E0463]: can't find crate for `serde_json``
- Ejemplo para `ImportPathFix`: `error[E0432]: unresolved import `crate::utils``

****Cálculo WEIGHT_COUNT:****

```
// crates/repair_nn_core/src/lib.rs const INPUT: usize = 64; // FeatureVector const
HIDDEN: usize = 32; // Capa oculta const LATENT: usize = 16; // Latent space const OPS:
usize = OPERATOR_COUNT; // 14 pub const WEIGHT_COUNT: usize = INPUT * HIDDEN + HIDDEN +
// Capa 1: 64×32 + 32 biases HIDDEN * LATENT + LATENT + // Capa 2: 32×16 + 16 biases
LATENT * OPS + OPS + // Capa 3: 16×14 + 14 biases LATENT + 1 + // Confidence head: 16×1
+ 1 bias LATENT + 1; // Risk head: 16×1 + 1 bias // = 64×32 + 32 + 32×16 + 16 + 16×14 +
14 + 16 + 1 + 16 + 1 = 2897
```

****Acción:****

- Actualizar `WEIGHT\_COUNT` en `repair\_nn\_core/src/lib.rs:148` (cambiar `2863` → `2897`)
- Propagar a: `ARCHITECTURE.md`, `PART2\_NEURAL\_NETWORK.md`, `INVENTORY.md`, `DISCREPANCIES.md`
- Añadir test: `assert\_eq!(WEIGHT\_COUNT, 2897)`

FASE 2 — Generador de Diff Determinista

****Problema:**** `apply()` devuelve `CandidatePatch{files, steps}` pero ****no genera diff real****.

****Solución:**** Implementar transformador por operador:

```
// crates/repair_operators/src/transform.rs pub fn transform( operator: OperatorId,
file_content: &str; , params: &BTreeMap; , ) -> Result { match operator {
OperatorId::DependencyRepair => { // Añadir dependencia a Cargo.toml let dep =
params.get("dependency").ok_or(TransformError::MissingParam)?; let version =
params.get("version").ok_or(TransformError::MissingParam)?;
add_dependency(file_content, dep, version) } OperatorId::ImportPathFix => { // Corregir
ruta de import let old_path = params.get("old").ok_or(TransformError::MissingParam)?;
let new_path = params.get("new").ok_or(TransformError::MissingParam)?;
fix_import_path(file_content, old_path, new_path) } // ... implementar para TODOS los
operadores _ => Err(TransformError::UnsupportedOperator), } } // Test por operador:
#[cfg(test)] mod tests { use super::*; #[test] fn test_dependency_repair_adds_dep() {
let toml = r#"[package] name = "test" [dependencies] "#; let mut params =
BTreeMap::new(); params.insert("dependency".to_string(), "serde".to_string());
params.insert("version".to_string(), "1.0".to_string()); let result =
transform(OperatorId::DependencyRepair, toml, &params).unwrap();
assert!(result.contains("serde = \"1.0\"")); } #[test] fn
test_import_path_fix_replaces() { let code = r#"use crate::utils::helper;"#; let mut
params = BTreeMap::new(); params.insert("old".to_string(),
"crate::utils::helper".to_string()); params.insert("new".to_string(),
"crate::helpers::utils".to_string()); let result =
transform(OperatorId::ImportPathFix, code, &params).unwrap(); assert_eq!(result, "use
crate::helpers::utils;"); } #[test] fn test_unsupported_operator_errors() { let result
= transform(OperatorId::Unknown, "", &BTreeMap::new()); assert!(matches!(result,
Err(TransformError::UnsupportedOperator))); } }
```

FASE 3 — Durable Object SQLite (Memoria)

****Esquema mínimo:****

```
-- wrangler.toml debe incluir: [durable_objects] bindings = [{ name = "INCIDENT_STATE",
class_name = "IncidentState" }] [[migrations]] tag = "v1" new_sqlite_classes =
["IncidentState"]
```

****Tablas:****

```
// worker/src/runtime/state.rs pub struct IncidentState { // SQLite via SqlStorage
(workers-rs 0.8) } // Esquema: // 1. incidents(id TEXT PRIMARY KEY, repository TEXT,
signature TEXT, // state TEXT, attempts INTEGER, created_at INTEGER, updated_at
INTEGER) // 2. idempotency(key TEXT PRIMARY KEY, response TEXT, created_at INTEGER) --
TTL // 3. transitions(id INTEGER PRIMARY KEY AUTOINCREMENT, incident_id TEXT, //
from_state TEXT, to_state TEXT, correlation_id TEXT, created_at INTEGER) // 4.
verification(incident_id TEXT PRIMARY KEY, status TEXT, evidence_ref TEXT, created_at
INTEGER) // 5. quota(key TEXT PRIMARY KEY, window_start INTEGER, count INTEGER)
```

****Cálculo de límites Free:****

- Límite: 100,000 filas escritas/día
- Por incidente: ~6 escrituras (incident + idempotency + 2 transitions + verification + quota)
- ****Techo: 100,000 / 6 ≈ 16,666 incidentes/día****

****Política de retención:****

- `transitions`: Borrar registros > 30 días (evita desborde)
- `idempotency`: TTL de 7 días
- `verification`: Mantener 90 días

FASE 4 — Modelo con Rollback Real

****Problema:**** `load\_weights` (worker/src/lib.rs:180) cae a `RepairNet::zeros()` si falla.

****Solución:****

```
// worker/src/runtime/model.rs pub async fn load_weights(env: &Env;) -> Result { // 1.
Intentar current.json if let Ok(weights) = load_from_kv(env, "current").await { if
validate_weights(&weights;) { return RepairNet::from_weights(&weights;) .map_err(|_|
ModelError::InvalidWeightCount); } } // 2. Fallback a stable.json if let Ok(weights) =
load_from_kv(env, "stable").await { if validate_weights(&weights;) { return
RepairNet::from_weights(&weights;) .map_err(|_| ModelError::InvalidWeightCount); } }
// 3. BLOCKED (NUNCA ceros) Err(ModelError::NoValidModel) } fn
validate_weights(weights: &[f32]) -> bool { weights.len() == WEIGHT_COUNT &&
weights.iter().all(|&w;| w.is_finite()) } // Test: #[cfg(test)] mod tests { use
super::*; #[test] fn test_corrupt_current_falls_to_stable() { // Simular current
corrupt, stable válido let current = vec![0.0; WEIGHT_COUNT - 1]; // Longitud
incorrecta let stable = vec![0.5; WEIGHT_COUNT]; // Válido // Mock KV: current → Err,
stable → Ok let result = load_weights_with_mock(|name| { if name == "current" {
Err(KVError::NotFound) } else if name == "stable" { Ok(stable.clone()) } else {
Err(KVError::NotFound) } }); assert!(result.is_ok()); } #[test] fn
test_both_corrupt_returns_blocked() { let result = load_weights_with_mock(|_|
Err(KVError::NotFound)); assert!(matches!(result, Err(ModelError::NoValidModel))); } }
```

****Estados de promoción:****

```
pub enum PromotionState { Candidate, Evaluating, Validated, Promoted, RolledBack,
Rejected, } pub struct PromotionRecord { pub promotion_id: String, pub dataset_version:
String, pub model_version: String, pub evaluation_results: BTreeMap, pub timestamp:
u64, pub state: PromotionState, }
```

****Regla:**** `training\_examples` solo se rellena con `verification == Pass` y `evidence\_ref` presente.

PROHIBICIONES ABSOLUTAS

- ■ No declarar `PASS`, `READY` ni `deployed` sin evidencia ejecutada

- ■ No inventar pesos, endpoints, IDs, tokens ni resultados de entrenamiento
- ■ No introducir MongoDB, D1, Queues, R2, Workflows, Workers AI, Mergify, Wolfram, Mem0
- ■ No reescribir `docs/NO\_LLM\_POLICY.md`
- ■ No ampliar `OperatorId` fuera del allowlist de FASE 1
- ■ No crear segunda autoridad de VERIFY (GitHub Actions es la única)

SALIDA REQUERIDA

1. Decisión por PR (MERGE/CLOSE/REBASE/BLOCKED) con justificación
2. Allowlist final + `WEIGHT\_COUNT` recalculado + propagación
3. Implementación de `transform()` por operador + tests
4. `wrangler.toml` completo + esquema SQLite + cálculo de escrituras/día
5. `load\_weights` con rollback + test
6. Cada comando ejecutado con salida real o `NOT RUN`

****Estado final:**** `READY` o `BLOCKED` con lista exacta de pendientes.